

KUBERNETES IN PRODUCTION · ISSUE 02 OF 08

HPA, VPA and Cluster Autoscaler.

Autoscaling is three controllers. Tune them together.

02

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

What happens in a spike

- 1 Traffic rises
- 2 HPA adds replicas
- 3 Pods go Pending
- 4 Cluster Autoscaler adds a node
- 5 Pods schedule

HPA utilization is measured against requests. No requests, no meaningful HPA.

Who does what

CONTROLLER	SCALES	BASED ON
HPA	Replica count	CPU, memory, custom metrics
VPA	Requests per pod	Observed usage
Cluster Autoscaler	Nodes	Unschedulable pods



Where they fight

01 HPA + VPA on CPU

Both react to the same signal. Use VPA in recommend mode.

02 Node lag

New nodes take minutes. Keep headroom and min replicas.

03 Scale-down disruption

Protect with PodDisruptionBudgets.

04 Flapping

Use stabilization windows.

HPA with sane behaviour

YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Scale up fast, scale down slowly.



Before you trust autoscaling

- ☒ Requests on every container
- ☒ Min replicas survive a node loss
- ☒ PDB on every Deployment
- ☒ Scale-up time load-tested
- ☒ Alert when maxReplicas is hit
- ☒ Node pool max sized for peak

KEY TAKEAWAY

Pods, requests and nodes scale as one system.

- HPA needs requests
- Do not let HPA and VPA fight over CPU
- Load-test the scale-up time

YOUR TURN

What broke the first time your cluster autoscaled under real load?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #03 Probes, PDBs and safe rollouts

Save · Repost

